5. Linear classification

Machine learning

Doc. Ing. Radim Kolar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Introduction

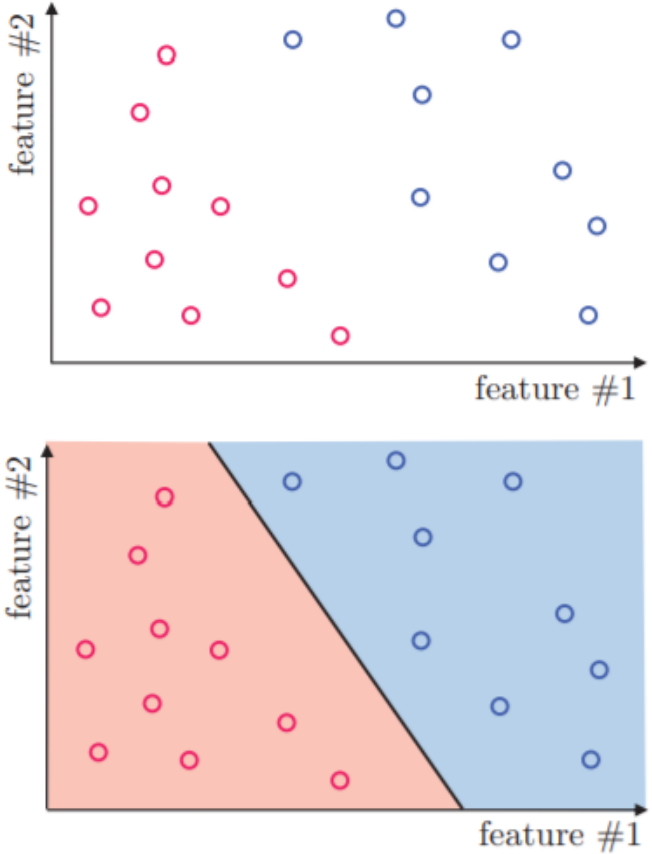
For linearly separated cases.

Linear classifier are simple and usually easy to compute (train).

Attractive candidates also for not strictly linearly separated cases… at least as a candidates for initial, trial classifier.

Offers simple generalization, which can be acceptable in some cases.

No danger of overfitting.



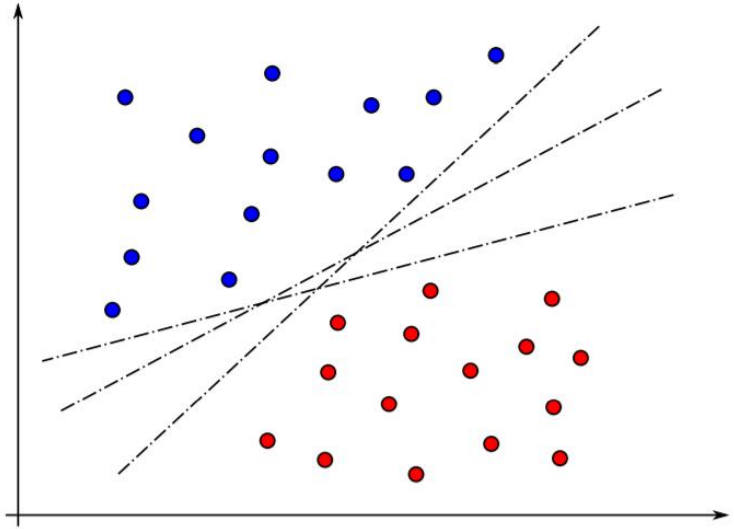
Introduction

Building a linear classifier - looking for linear **hyperplane** (or more general as **separating/decision hyperplane/hypersurface** or **discriminant function**) that separates two or more classes by a linear function - linear in the components of feature vector **x**.

In 2D case - the linear function is a line.

What is the best “position” of this line?

We need some criteria (i.e. cost function), in order to optimize the position.



Linear hyperplane

Consider now a two-class problem. The respective separating hyperplane in the n-dimensional feature space can be mathematically described by equation:

$$g(\mathbf{x}) = \mathbf{w'^T x'} + b = 0,$$

where $\mathbf{w'} = [w'_1, w'_2, \ldots w'_n]$ is a known weight vector and *b* is a *threshold*.

For a 2D feature vector we can write an equation for a line:

$$g(x_1, x_2) = w'_1 x'_1 + w_2 x'_2 + b = 0$$

Compare it with equation for a line in 2D: a.x + b.y + c = 0, or y = -(a/b).x - c/b

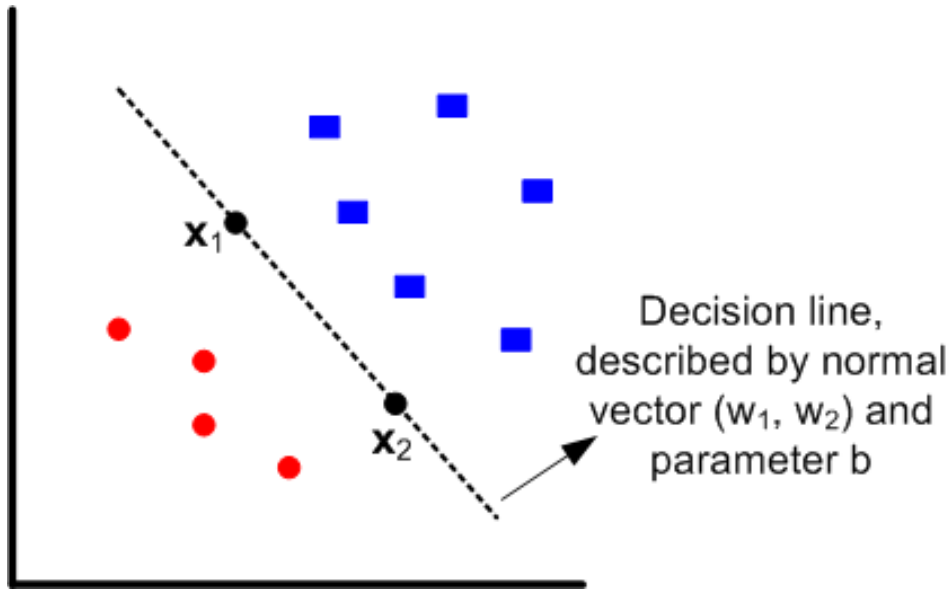
Linear hyperplane

Consider two points $\mathbf{x}_1$ and $\mathbf{x}_2$ on the line, then the following is valid:

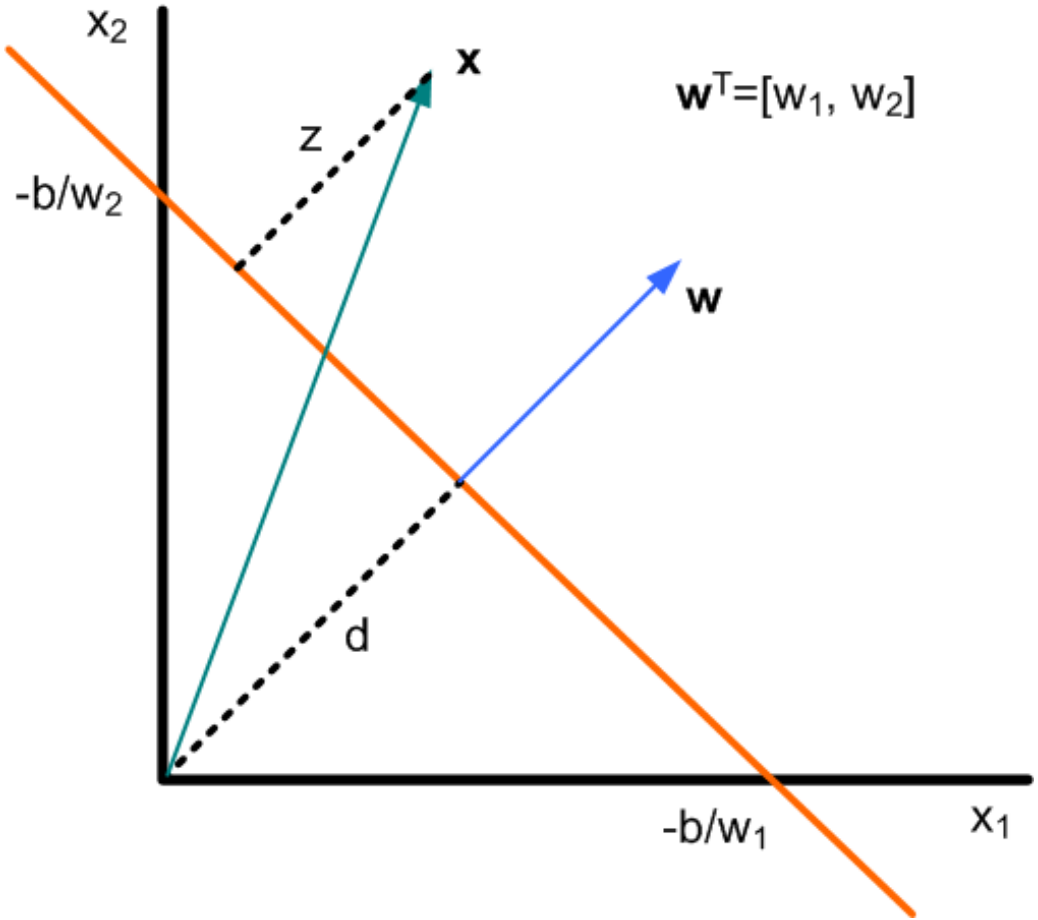
$$0 = \mathbf{w'}^T \mathbf{x'}_1 + b = \mathbf{w'}^T \mathbf{x'}_2 + b$$

$$\mathbf{w'}^T (\mathbf{x'}_1 - \mathbf{x'}_2) = 0$$

Since the vector $(\mathbf{x'}_1 - \mathbf{x'}_2)$ obviously lies on the decision line, it is apparent that $\mathbf{w'}^T$ is orthogonal to this line and therefore it is its *normal vector*.



Linear hyperplane



$\mathbf{w}^T=[w_1, w_2]$

It can be shown (see some math book) that the distance of the line from origin is:

$$d = \frac{|b|}{\sqrt{w_1^2+w_2^2}}$$

and the distance of some point **x** to the line is:

$$z = \frac{g(\mathbf{x})}{\sqrt{w_1^2+w_2^2}}$$

It also holds that *z* value on one plane side the g(x) takes positive values and on the other negative. If we want a *real distance* we need to take |g(x)| in the numerator.

g(x) represents function of a line (or generally some hyperplane), e.g.

g(x): $w_1 x_1 + w_2 x_2 + b = 0$

('standard equation: a.x+b.y+c=0')

Cost function

A cost function $J(\mathbf{x},y,\mathbf{w})$ quantifies how unhappy you would be if you used **w'** (including b) to make a classification of vector **x** when the correct output is y.

Therefore, we want to minimize $J(.)$.

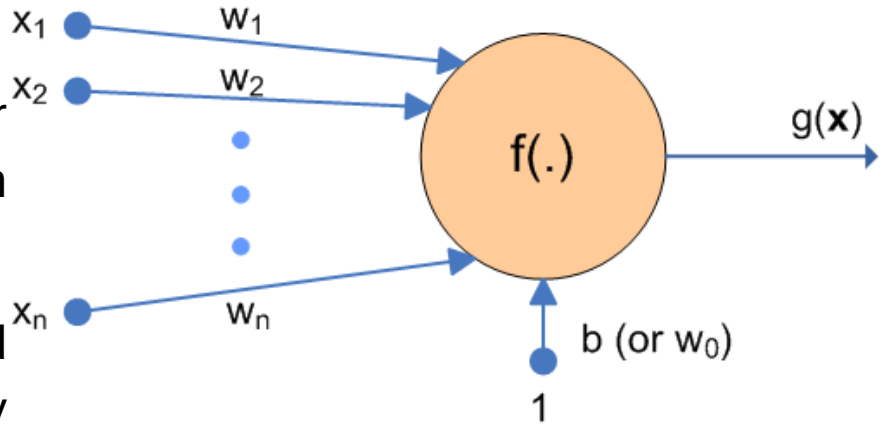
There are different versions of the cost function. We will closely examine these:

- Perceptron
- Mean squared error
- Fisher linear discriminant
- Support vector

Perceptron Cost Function

Perceptron

The simple math description of the linear hyperplane is equivalent to perceptron model.



Now we will describe the training based perceptron algorithm. Before that we slightly modify the notation:

$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ : \\ x_n \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ : \\ w_n \end{bmatrix}$$

With this notation we can simply write the separating hyperplane as:

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = 0$$

So, the w_0 is included in vector **w** and we must also include 1 into vector **x**. These vectors are sometimes called as *augmented vectors*.

Perceptron algorithm

The cost function in classical perceptron algorithm considers only misclassified samples.

We introduce labels for two-class problem, with classes $\boldsymbol{\omega}_1$ and $\boldsymbol{\omega}_2$. The labels are $\ell_x = \pm 1$. Let's consider weight of <u>correctly trained</u> classifier $\mathbf{w}$ such that:

$$\mathbf{w}^T \mathbf{x} > 0 \quad \forall\, \mathbf{x} \in \boldsymbol{\omega}_1 \text{ (we set the labels } \ell_x = +1 \text{ for this class)}$$

$$\mathbf{w}^T \mathbf{x} < 0 \quad \forall\, \mathbf{x} \in \boldsymbol{\omega}_2 \text{ (we set the labels } \ell_x = -1 \text{ for this class)}$$

Shortly: if the sample belongs to class $\boldsymbol{\omega}_1$ then the output of classifier is positive and *vice versa*.

Perceptron method

The criterium can be then formulated as:

$$J_{Perc}(\mathbf{w}) = -\sum_{x \in Y_{miss}} l_x \mathbf{w}^T \mathbf{x}$$

where Y_{miss} is a set of samples misclassified by **w**. If no samples are misclassified, the Y_{miss} is empty and we define J_P to be zero.

Note that J_{Perc}:

- is alway positive
- geometrically, it is proportional to the sum of distances of the misclassified samples from decision boundary.
- is piecewise constant.

Perceptron method

Now, if we want to apply gradient descent method for minimizing this function, we must compute the gradient of J_{Perc} with respect to **w**:

$$\frac{\partial J_{Perc}(\mathbf{w})}{\partial \mathbf{w}} = -\sum_{x \in Y_{miss}} l_x \mathbf{x}$$

Let's look closer on 2D case and make the derivatives for each components of **w**. Remind that $g(x_1, x_2) = w_1 x_1 + w_2 x_2 + b = 0$.

$$\frac{\partial J_{Perc}(\mathbf{w})}{\partial b} = -\sum_{x \in Y_{miss}} l_x$$

$$\frac{\partial J_{Perc}(\mathbf{w})}{\partial w_1} = -\sum_{x \in Y_{miss}} l_x x_1$$

$$\frac{\partial J_{Perc}(\mathbf{w})}{\partial w_2} = -\sum_{x \in Y_{miss}} l_x x_2$$

Perceptron method

Gradient descent take the form:

$$\mathbf{w(t+1)} = \mathbf{w(t)} - \rho_t(-\sum_{x \in Y_{miss}} l_x \mathbf{x})$$

The learning rate ρ_t may be constant or depending on iteration *t*.

We will omit the proof of convergence of this method. But it will converge only for linearly separable cases.

Perceptron method

Practical limitations of this approach:

- Convergence may be slow.
- If the data are not separable, the algorithm will not converge.
- We will only know that the samples are separable once the algorithm converges.
- The solution is in general not unique, and will depend upon initialization (**w**) and the learning rate ρ_t.

Modifications of perceptron algorithm

There are various modifications of this basic perceptron algorithm, e.g.

Pocket Algorithm

1. Run the training and compute $\mathbf{w}_t$.
2. Test the classifier - evaluate classification error.
3. If this error is smaller than previous one, then store (*in the pocket*) current $\mathbf{w}_t$.
4. In such a way, we will get the optimal solution also for non-separable set.

Mean Square Error
Cost Function

Mean square error (MSE) method

The linear separation of classes is usually not very often - in practice the samples from different class may slightly overlap. And also in these cases, it is still worth to use linear classification. But we have to live with error.

We will attempt to design a linear classifier so that its desired output is again ±1, depending on the class ownership of the input vector. But due to possible classification error, the predicted output will not always be equal to desired one.

Given an input vector **x**, the output of the classifier will be $\mathbf{w}^T\mathbf{x}$ (threshold can be added to have ±1 output). Then, the weight vector **w** will be computed so as to minimize the mean square error (MSE) between the desired (y_i) and true output:

$$J_{MSE}(\mathbf{w}) = \sum_{i=1}^{p} (y_i - \mathbf{w}^T \mathbf{x_i})^2$$

MSE method

This might be rather weird to compare “some distance” with labels. But it works. Let’s elaborate on this:

Class with label +1, i.e. $(+1 - w^t x)^2$

Correct classification: $(1 - (+k))^2$ ↓

Incorrect classification: $(1 - (-k))^2$ ↑

Class with label -1, i.e. $(-1 - w^t x)^2$

Correct classification: $(-1 - (-k))^2$ ↓

Incorrect classification: $(-1 - (+k))^2$ ↑

$$J_{MSE}(\mathbf{w}) = \sum_{i=1}^{n} (y - \mathbf{w}^T \mathbf{x})^2$$

MSE method

The criterion can be written also in an alternative way, with matrix notation and L^2 norm ($||\ .\ ||^2$):

$$J_{MSE}(\mathbf{w}) = \sum_{i=1}^{p}(y_i - \mathbf{w}^T\mathbf{x_i})^2 = ||\mathbf{y} - \mathbf{X}\mathbf{w}||^2$$

where **y** is the vector of labels and **X** is a matrix of (augmented) input vectors - each row is an augmented input vector.

Again, for gradient-based minimization we need to compute the gradient:

$$\nabla J_{MSE}(\mathbf{w}) = \sum_{i=1}^{p} -2(y_i - \mathbf{w}^T\mathbf{x_i})\mathbf{x_i} = -2\mathbf{X^T}(\mathbf{y} - \mathbf{X}\mathbf{w})$$

MSE method

Here, we can set the gradient to zero:

$$2\mathbf{X}^{\mathbf{T}}(\mathbf{y} - \mathbf{X}\mathbf{w}) = 0$$

We are looking for **w**:

$$\mathbf{X}^{\mathbf{T}}\mathbf{X}\mathbf{w} = \mathbf{X}^{\mathbf{T}}\mathbf{y}$$

And finally:

$$\mathbf{w} = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y} = \mathbf{X}^{\dagger}\mathbf{y}$$

This is a direct solution, which needs the inverse matrix of $\mathbf{X}^{\mathbf{T}}\mathbf{X}$, which is square and often nonsingular.

MSE method - example

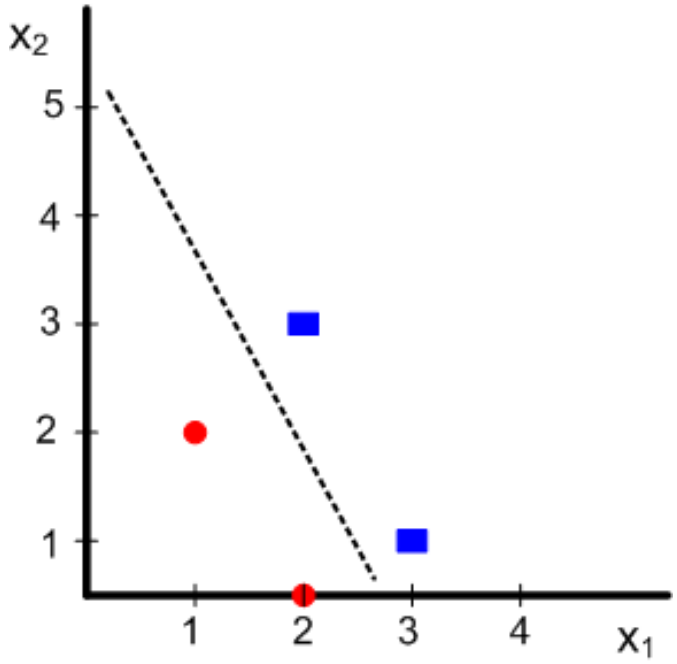
Suppose we have four samples.

Class +1: $(1,2)^T$, $(2,0)^T$

Class -1: $(3,1)^T$, $(2,3)^T$

Let's create (augmented) matrix **X** and prepare vector **y**:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 0 \\ 1 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}$$



MSE method - example

Now compute the $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$:

$$\mathbf{X}^\dagger = (\mathbf{X}^\mathbf{T}\mathbf{X})^{-1}\mathbf{X}^\mathbf{T} = \begin{bmatrix} 1.250 & 1.083 & -0.750 & -0.583 \\ -0.500 & -0.167 & 0.500 & 0.167 \\ 0 & -0.333 & 0 & 0.3333 \end{bmatrix}$$

and the weight vector is computed as:

$$\mathbf{w} = \mathbf{X}^\dagger \mathbf{y} = \begin{bmatrix} 1.250 & 1.083 & -0.750 & -0.583 \\ -0.500 & -0.167 & 0.500 & 0.167 \\ 0 & -0.333 & 0 & 0.3333 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3.67 \\ -1.33 \\ -0.67 \end{bmatrix}$$

MSE method

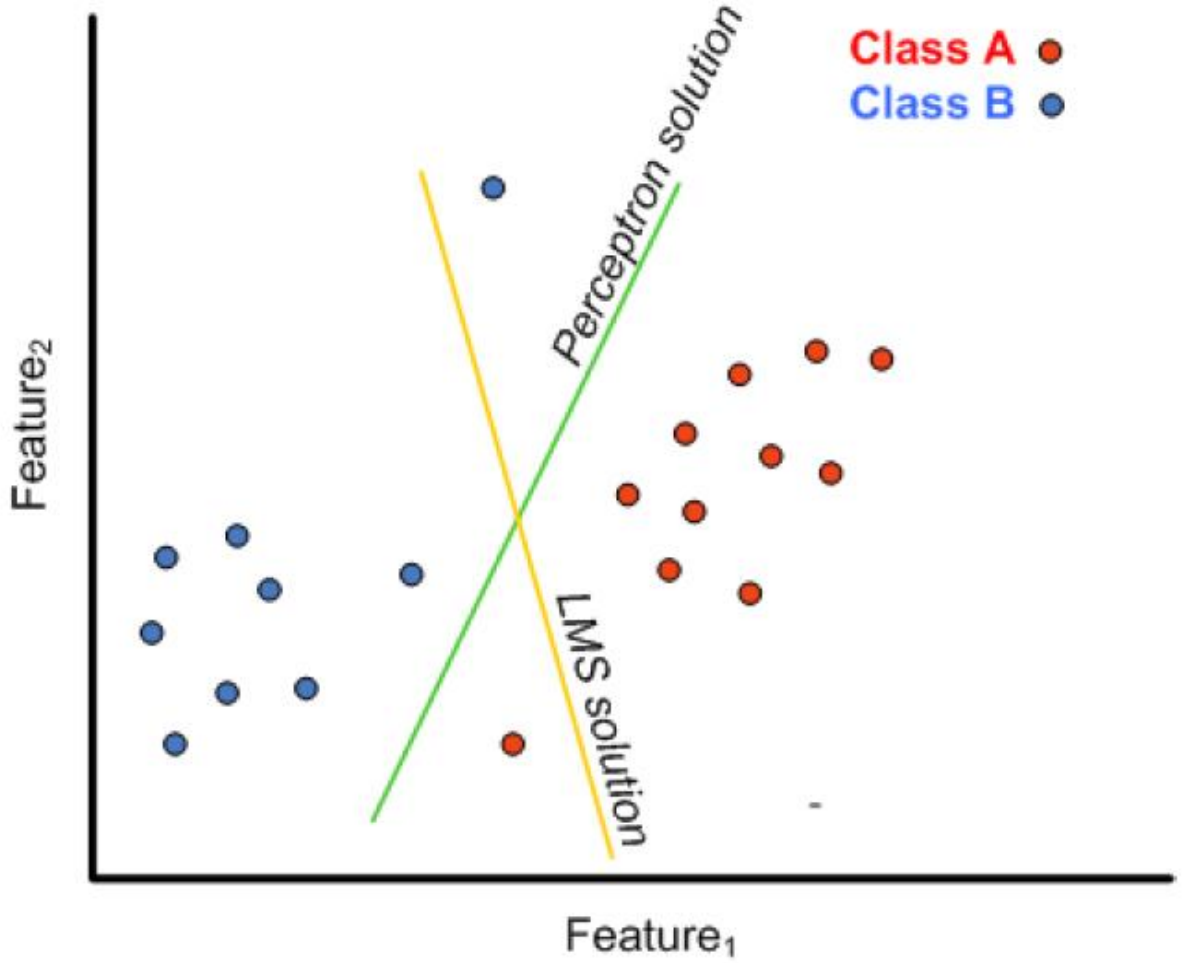
The simple *inverse matrix technique* is sometime not possible. Therefore the iterative optimization gradient descent scheme is applied, similarly as in perceptron case:

$$\mathbf{w(k+1)} = \mathbf{w(k)} + \rho_k (y_k - \mathbf{w(k)}^{\mathbf{T}} \mathbf{x_k}) \mathbf{x_k}$$

Note that in this case the index *k* for updating the parameter (**w**) coincides with the index of the current input sample $\mathbf{x_k}$. So, this is a *time-adaptive* scheme, which adapts to the solution as successive samples become available to the system. This algorithm is known as a least mean squares (LMS) or Widrow-Hoff algorithm.

This method can be of course seen as training the neuron without (generally) nonlinear activation function which is known as *adaline* (adaptive linear element).

Comparison LMS perceptron metho



MSE + L^2 regularization

MSE can become unstable (during training the solution is oscillating around the optima). Regularization is a general method to avoid these problems by imposing additional constraints on weight vector **w**. A common approach is to make sure that the weights are, on average, small in magnitude; this is also referred as a *shinkage*.

The regularized version is:

$$J_{ridge}(\mathbf{w}) = \sum_{i=1}^{p}(y - \mathbf{w}^T\mathbf{x})^2 + \lambda||\mathbf{w}||^2 = ||\mathbf{y} - \mathbf{X}\mathbf{w}||^2 + \lambda||\mathbf{w}||^2$$

where $||\mathbf{w}||^2 = \sum w_i^2$ is the squared norm of **w** or equivalently, the dot product $\mathbf{w}^T\mathbf{w}$, λ is scalar value determining the level of regularization. This L^2 norm regularization is often called as **ridge** regularization.

MSE + L^2 regularization

This form of regularization still have a closed form solution:

$$\mathbf{w} = (\mathbf{X^T X} + \lambda \mathbf{I})^{-1} \mathbf{X^T y}$$

where $\mathbf{I}$ is identity matrix. If we compare this equation with equation for **w** without regularization, we see that here we add λ to the diagonal of $\mathbf{X^T X}$, which is know trick to improve the stability of matrix inversion. This form of is also known as ridge classification or regression.

MSE + L^1 regularization

Another way of regularization is the absolute value of **w** coefficients:

$$J_{lasso}(\mathbf{w}) = \sum_{i=1}^{p} (y - \mathbf{w}^T \mathbf{x})^2 + \lambda |\mathbf{w}| =$$

$$||\mathbf{y} - \mathbf{X}\mathbf{w}||^2 + \lambda |\mathbf{w}|$$

where we sum the absolute values of the weights $|\mathbf{w}| = \sum |w_i|$. The result is that some weights are shrunk and others are set to zero. In other words - some features are set as less important or totally unimportant in our classification task. This regularization is called as lasso ("least absolute shrinkage and selection operator"). It is rather sensitive to regularization parameter, which is usually set during cross-validation. More information about ridge and lasso regularization: https://www.youtube.com/watch?v=sO4ZirJh9ds

Fisher Linear Discriminant Classifier

Fisher’s Linear Discriminant

Linear discriminant analysis (LDA) discussed in Feature reduction part can be used also for classification. Just set some threshold (e.g. compute ROC curve and set threshold according to desired classifier properties).

Fisher’s Linear Discriminant, in essence, is a technique for dimensionality reduction, not a discriminant.

To find the optimal direction to project the input data, Fisher needs supervised data.

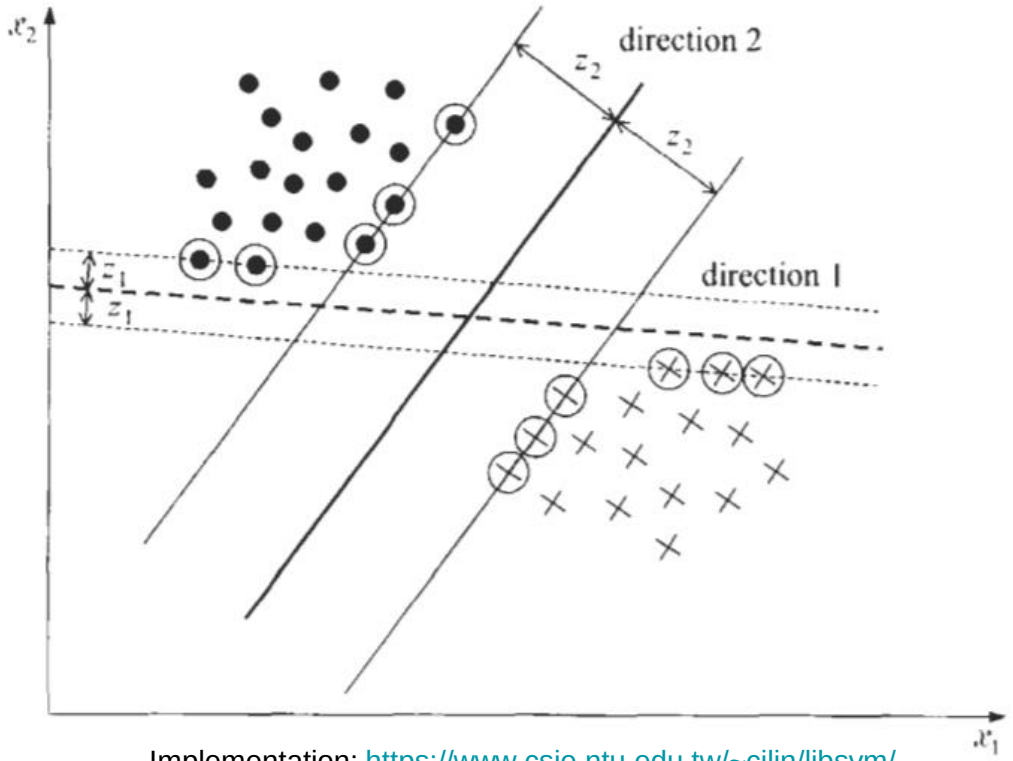
Given a dataset with D dimensions, we can project it down to D-1, D-2… up to 1 dimension.

Support vector classification (*Support vector machine*)

Support vector linear classification

Suppose a linearly separable two class case. The sample in these classes are separated (in a feature space) by “some space”, sometimes called as a *buffer zone*.

But there can be more buffer zones. The figure shows two options. Which one would you select?



Implementation: https://www.csie.ntu.edu.tw/~cjlin/libsvm/

Linear SVM

Again, the goal is to find a hyperplane

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = 0$$

that classify correctly all training vectors. Now our task is to find direction of separating hyperplane, that gives us the maximum possible margin (or buffer zone).

From math we know that the distance of a point **x'** to line (or hyperplane) is given by:

$$z = \frac{|g(\mathbf{x}')|}{||\mathbf{w}||}$$

*Note: In SVM is better to work with **w** and b separately - we will now consider the non-augmented vectors, which makes the analysis easier.*

Linear SVM

We can now scale **w** so that the value of g(**x'**) at the nearest points in both classes is equal to +1 or -1, respectively. (i.e. g(**x'**)=±1). This is not a restriction. We just simplify the next analysis. This is then equivalent with:

1. The margin size (buffer zone width):

$$\frac{1}{||\mathbf{w}||} + \frac{1}{||\mathbf{w}||} = \frac{2}{||\mathbf{w}||}$$

2. Requiring that all samples will be on or outside the margin:

$$\mathbf{w^T x_i} + b \geq 1, \;\; \forall \mathbf{x_i} \in class\ with\ labels\ y_i = +1$$
$$\mathbf{w^T x_i} + b \leq -1, \;\; \forall \mathbf{x_i} \in class\ with\ labels\ y_i = -1$$

These two equations can be written as one (with the help of the labels y_i):

$$y_i \left(\mathbf{w^T x_i} + b \right) \geq 1, \forall \mathbf{x_i}$$

In SVM the labels are typically marked as y

Linear SVM

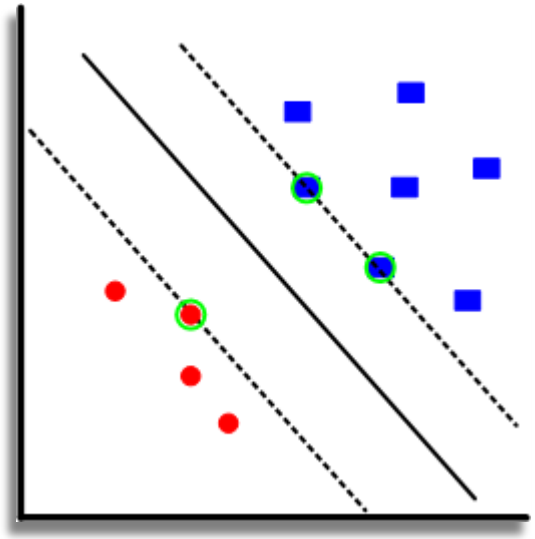
Suppose that we already trained classifier, so we have **w** and b. Then this inequality will hold for all samples:

$$y_i\left(\mathbf{w^T x_i} + b\right) \geq 1, \forall \mathbf{x_i}$$

But in which cases it become to equality?

$$y_i\left(\mathbf{w^T x_i} + b\right) = 1$$

This equality is valid for the samples lying on the margin of the trained classifier. We call this samples as **support vectors**. The number of support vectors depends on “spatial distribution” of the samples in the training set.



Linear SVM

Because the margins are $\frac{2}{||\mathbf{w}||}$, then if we minimize ||**w**||, we also maximize margins (i.e. buffer zone).

The principle of this method can be summarized as (using square root of ||**w**||):

$$minimize \;\; J_{SVM}(\mathbf{w}) = \frac{1}{2}||\mathbf{w}||^2$$

$$subject\ to\ y_i(\mathbf{w^T x_i} + b) \geq 1, \forall \mathbf{x_i}$$

This is non-linear (quadratic) optimization task subject to a set of linear inequality constraints, which can be solved by Lagrange multipliers.

Linear SVM

This objective function with inequality constraints form a so-called constrained optimization problem, which is often solved by Lagrange multipliers and Lagrangian:

$$L(\mathbf{w}, b, \lambda) = \frac{1}{2}||\mathbf{w}||^2 - \sum_i \lambda_i [y_i(\mathbf{w^T x_i} + b) - 1]$$

This Lagrangian has to be **minimized** with respect to primal variables ***w*** and *b* and **maximized** with respect to dual variables λ_i ($\lambda_\mathbf{i} \geq 0 \;\; for\ all\ i = 1, .., p$)

Differentiation of ***L(.)*** with respect to primal variables gives these conditions:

$$\frac{\partial L(\mathbf{w},b,\lambda)}{\partial b} = 0 \;\; and \;\; \frac{\partial L(\mathbf{w},b,\lambda)}{\partial \mathbf{w}} = 0$$

Leads to:

$$\sum_i \lambda_i y_i = 0 \;\; and \;\; \mathbf{w} = \sum_i \lambda_i y_i \mathbf{x_i}$$

Linear SVM

By substituting these conditions into Lagrangian, we eliminate the primal variables and we get so-called *dual optimization problem*, which is usually solved in practice:

$$L(\lambda) = \sum_{i=1}^{p} \lambda_i - \frac{1}{2} \sum_{i=1}^{p} \sum_{j=1}^{p} \lambda_i \lambda_j y_i y_j \mathbf{x_i^T x_j}$$

So, we **maximize** this $L(\boldsymbol{\lambda})$ with respect to $\boldsymbol{\lambda}$ and subject to:

$$\sum_i \lambda_i y_i = 0$$

and

$$\lambda_\mathbf{i} \geq 0 \;\; for\ all\ i = 1, .., p$$

Linear SVM

The optimization can be solved by some appropriate “tool”. Often, a quadratic programming package is used. Because these packages usually search for minima (and not for maxima as in our case), we need to change the sign. The original expression is

$$\max_{\lambda} \sum_{i=1}^{p} \lambda_i - \frac{1}{2} \sum_{i=1}^{p} \sum_{j=1}^{p} \lambda_i \lambda_j y_i y_j \mathbf{x_i^T x_j}$$

and we change it to:

$$\min_{\lambda} \frac{1}{2} \sum_{i=1}^{p} \sum_{j=1}^{p} \lambda_i \lambda_j y_i y_j \mathbf{x_i^T x_j} - \sum_{i=1}^{p} \lambda_i$$

dot product between samples

Linear SVM

$$\lambda = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ : \\ \lambda_p \end{bmatrix}$$

Let’s write the last expression in a form of matrices:

$$\min_{\lambda} \ \frac{1}{2}\lambda^{\mathbf{T}} \begin{bmatrix} y_1 y_1 \mathbf{x_1^T x_1} & y_1 y_2 \mathbf{x_1^T x_2} & \dots & y_1 y_p \mathbf{x_1^T x_p} \\ y_2 y_1 \mathbf{x_2^T x_1} & y_2 y_2 \mathbf{x_2^T x_2} & \dots & y_2 y_p \mathbf{x_2^T x_p} \\ . & . & \dots & . \\ y_p y_1 \mathbf{x_p^T x_1} & y_p y_2 \mathbf{x_p^T x_2} & \dots & y_p y_p \mathbf{x_p^T x_p} \end{bmatrix} \lambda + (\mathbf{-1})^{\mathbf{T}} \lambda$$

subject to linear constraint:

$$\mathbf{y^T} \lambda = 0$$

and range for Lagrange multipliers:

$$0 \leq \lambda < \infty$$

To remind:
x - feature of our training set
y - labels of our training set
p - number of datapoints
n - number of features

SVM linear

Note the dimensions of the matrix on the previous slide - it depends on the number of samples. If we work with hundreds, or few thousands of samples we are moreless OK. But for higher number of samples it becomes more tricky.

The result of solution is the vector of Lagrange coefficients, $\boldsymbol{\lambda}$. Those $\mathbf{x_i}$ for which $\lambda_i>0$ are called support vectors.

But we need parameters of our classifier. We need to find ***w*** and *b*. For **w**, we can use the equation of the constraint:

$$\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x_i}$$

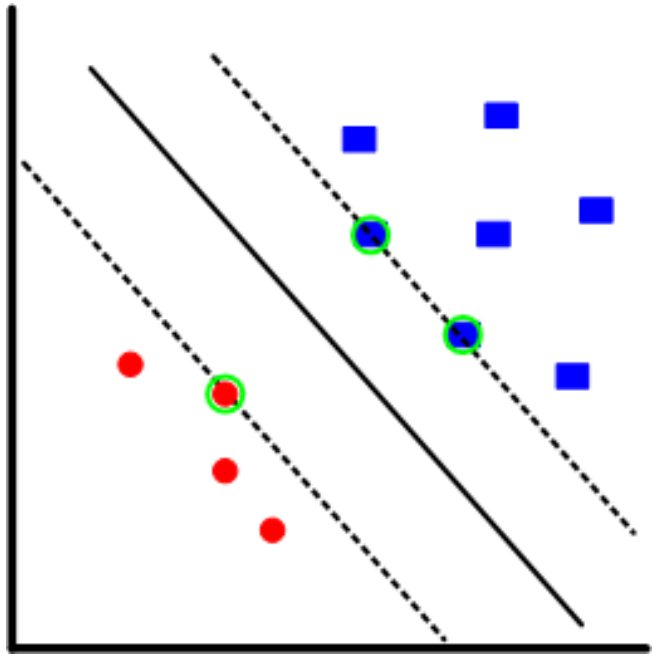
How to solve for b?

Linear SVM

We know that at the margin are support vectors, i.e. some of the training samples. And we also know what is the equation for the margins:

$$y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x_i} + b) = 1$$

Therefore, we solve for *b* using this equation and any support vector (for any $\mathbf{x_i}$ with $\lambda_i>0$).



Evaluation of the new sample

The discriminant function is given as

$$h(\mathbf{x}) = \sum_{\mathbf{i} \in \mathbf{SV}} \lambda_\mathbf{i} \mathbf{y_i} \mathbf{x_i^T} \mathbf{x} + \mathbf{b}$$

Thus we take new sample **x** and compute the value of above formula. If it is positive, then it belongs to “+1 class” if it is negative then it belong to “-1 class”.

For computation we use all the support vectors (SV), i.e. the samples from training set that defines the margin.

Linear SVM, non-separable case, soft margin

When the classes are not separable, the above setup is no longer valid. In this non-separable case we can have three cases:

- Vectors that fall outside the buffer zone and satisfy:

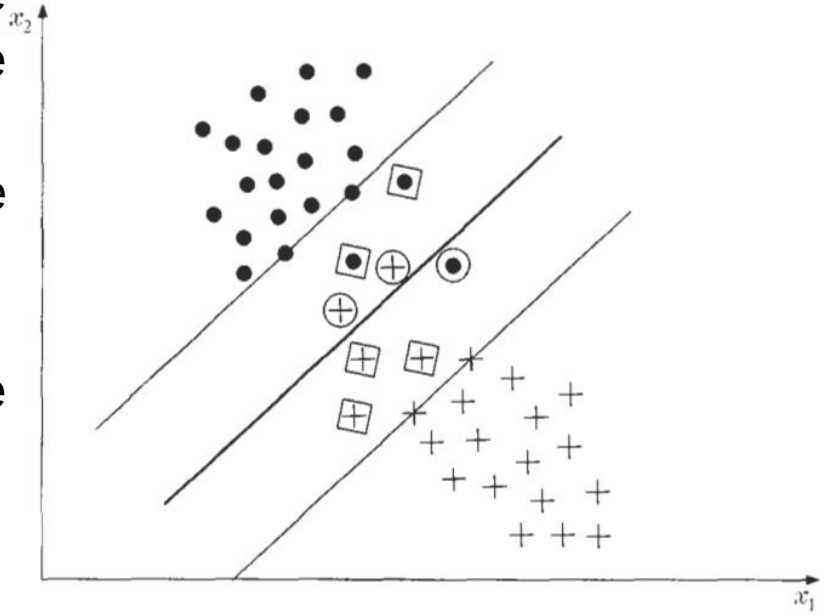
$$y_i(\mathbf{w}^T\mathbf{x}_i+b) > 1$$

- Vectors falling inside the zone and are correctly classified:

$$0 \leq y_i(\mathbf{w}^T\mathbf{x}_i+b) < 1$$

- Vectors that are misclassified:

$$y_i(\mathbf{w}^T\mathbf{x}_i+b) < 0$$



Soft margin, C-SVM

All three cases can be treated under a single type of constraints by introducing a new set of variables:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$$

The first category of data corresponds to $\xi_i = 0$, the second category to $0 < \xi_i \leq 1$, and the third to $\xi_i > 1$. The variables ξ_i are known as a *slack variables*. The optimization task is now more complex - to make the margin as large as possible but at the same time to keep the number of points with $\xi_i>1$ as small as possible. So we can formulate the new cost function for minimization:

$$J_{C-SVM}(\mathbf{w}, b, \xi) = \frac{1}{2}||\mathbf{w}||^2 + C\sum_i \xi_i$$

subject to:

$$y_i(\mathbf{w}^\mathbf{T}\mathbf{x_i} + b) \geq 1 - \xi_i, \quad and \quad \xi_i \geq 0 \quad i = 1, 2, \ldots p$$

Soft margin, C-SVM

If we now compare original SVM with this C-SVM and solution based on Lagrangian, you get almost the same solution (without proof) :

$$L(\lambda) = \sum_{i=1}^{p} \lambda_i - \frac{1}{2} \sum_{i=1}^{p} \sum_{j=1}^{p} \lambda_i \lambda_j y_i y_j \mathbf{x_i^T x_j}$$

So, we **maximize** this *L(**λ**)* with respect to ***λ*** and subject to:

$$\sum_i \lambda_i y_i = 0$$

and

$$0 \leq \lambda_\mathbf{i} \leq C \;\; for\ all\ i = 1, .., p$$

Soft margin, C-SVM

C value is a constant, which defines (set by user) the relative importance of the two terms in $J_{C\text{-}SVM}$.

How to set C? Look again:

$$J_{C-SVM}(\mathbf{w}, b, \xi) = \frac{1}{2}||\mathbf{w}||^2 + C\sum_i \xi_i$$

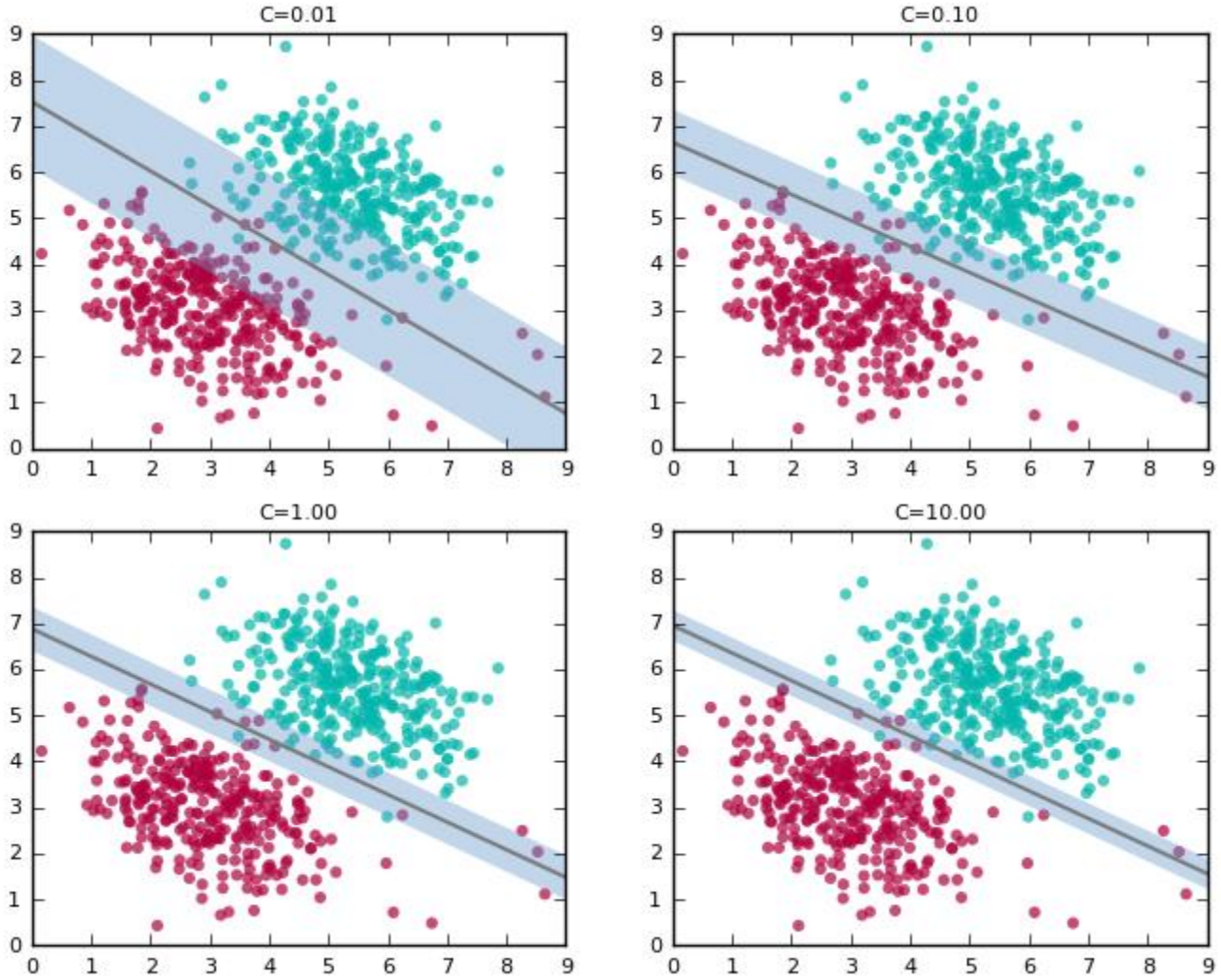
If C is very very high - a small error (i.e. slack or violation of the margin) would give high value of the second term, which leads to almost previous solution.

If C is very very small we can extend the margin and we get a lot of samples lying inside the margin.

Based on C we can compromise; C is set based on intuition, trial-error.

Soft margin, C-SVM

Example of C values.



ν-SVM

In the previous method, we've seen that there exists close connection between parameter C and width of the margin obtained as a result of optimization. However, the margin will depend on the number of samples and *amount* of overlapping. Since the margin is important (it is an essence of SVM, after all), we would like to introduce some parameter that would be directly connected to margin width. This margin can be simply defined by the pair of hyperplanes:

$$\mathbf{w}^T\mathbf{x} + b = \pm\varrho$$

where $\varrho \geq 0$ is a variable to be optimized. Under this new setting, the problem can be rewritten with the new cost function.

Soft margin, $\boldsymbol{\nu}$-SVM

This cost function is minimized:

$$J_{\nu-SVM}(\mathbf{w}, b, \xi, \rho) = \tfrac{1}{2}||\mathbf{w}||^2 - \nu\rho + \tfrac{1}{p}\textstyle\sum_i \xi_i$$

subject to

$$\begin{aligned} y_i(\mathbf{w}^{\mathbf{T}}\mathbf{x_i} + b) &\geq \rho - \xi_i, \quad i = 1, 2, \ldots p \\ \xi_i &\geq 0 \qquad\quad i = 1, 2, \ldots p \\ \rho &\geq 0 \end{aligned}$$

Here $\boldsymbol{\nu}$ represents an upper bound for number of errors (and hence also on the fraction of training errors) and also lower bound of the fraction of support vectors; i.e. if $\boldsymbol{\nu}$=0.05 there will be a most 5% of training samples miss-classified and at least 5% of training samples will be support vectors.

Non-linear (kernel) SVM

Beyond the linearity

The linear classifiers are simple but not always sufficient for classification task. Here we will introduce the non-linear classifiers based on the kernel approach. Yes, the kernel trick as in nonlinear PCA will be used again here.

The main idea is simple: to transform the data non-linearly to a feature space in which the linear classification can be applied.

Beyond the linearity

Now we will call the transformed space a feature space and the original space as input space. The approach is:

- transform the data from input space to feature space (typically with higher number of dimensions)
- train and evaluate linear classifier
- in order to classify new samples, we must first transform the new sample into feature space and then perform classification

But - the big thing is that the feature space does not have to be explicitly constructed, because we can perform all necessary operation in input space.

Kernel trick

In many machine learning methods, the dot product over the data is applied. The results of the dot product of two vectors is a scalar value. So, consider the dot product of two vectors (in 2 dimensions):

$$\mathbf{x_i^T x_j} = \langle \mathbf{x_i}, \mathbf{x_j} \rangle = \langle \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix} \rangle = x_{i1}x_{j1} + x_{i2}x_{j2}$$

Now, consider the dot product after transforming the features into higher dimension (from previous example):

$$\langle \phi(\mathbf{x_i}), \phi(\mathbf{x_j}) \rangle = \langle \begin{bmatrix} x_{i1}^2 \\ x_{i2}^2 \\ \sqrt{2}x_{i1}.x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1}^2 \\ x_{j2}^2 \\ \sqrt{2}x_{j1}.x_{j2} \end{bmatrix} \rangle = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

Kernel trick

The disadvantage of this approach is that we first need to transform all the feature set into high dimension. And because we typically have many features (not only two) it creates very high dimensional space (p>>m) and therefore it is computationally demanding and also the memory storage increases.
Fortunately, we can do it in a smarter way:

$$\mathbf{x_i^T x_j} = \langle \mathbf{x_i}, \mathbf{x_j} \rangle^2 = \langle \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix} \rangle^2 =$$

$$(x_{i1}x_{j1} + x_{i2}x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

So, we are able to compute the dot product **without** explicitly going to higher dimension!

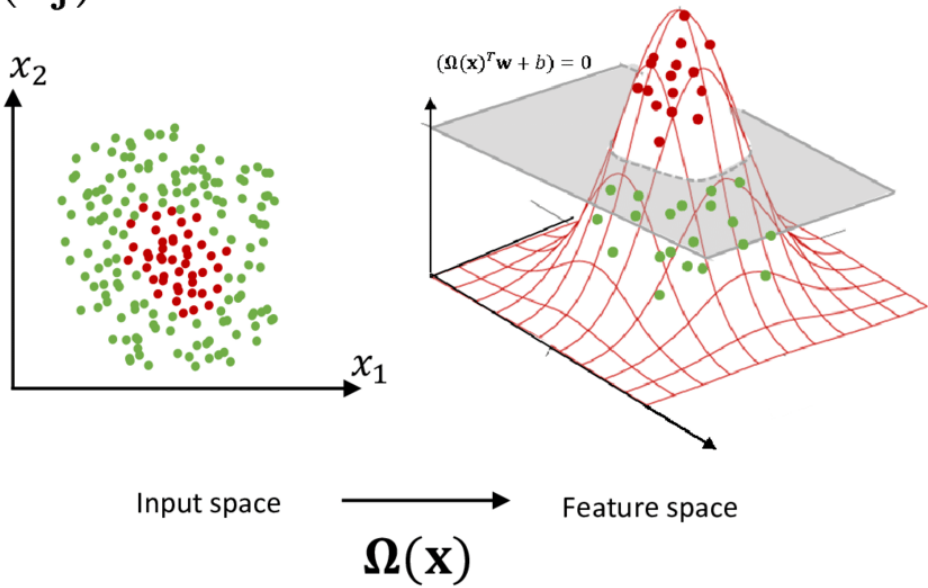
Kernel trick

To conclude - the kernel trick can be written as:

$$K(\mathbf{x_i}, \mathbf{x_j}) = \langle \phi(\mathbf{x_i}), \phi(\mathbf{x_j}) \rangle = \phi(\mathbf{x_i^T}) \phi(\mathbf{x_j})$$

This is sometimes called as generalized dot product.

The examples of various kernels are on the next slides.



https://www.researchgate.net/figure/Figure-B16-Non-linear-classifier-using-Kernel-trick_fig13_324250451

Beyond the linearity

Recall that linear SVM maximizes this term:

$$\max_{\lambda} \left(\sum_i \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x_i^T x_j} \right)$$

The important is the inner product $\mathbf{x_i^T x_j}$ (or written as $\langle \mathbf{x_i}, \mathbf{x_j} \rangle$). As we have seen we can apply “kernel trick” on this inner product.

we apply kernel K(.,.) on each matrix element (i.e. each dot product).

$$\min_{\lambda} \ \frac{1}{2} \lambda^{\mathbf{T}} \begin{bmatrix} y_1 y_1 \mathbf{x_1^T x_1} & y_1 y_2 \mathbf{x_1^T x_2} & \dots & y_1 y_p \mathbf{x_1^T x_p} \\ y_2 y_1 \mathbf{x_2^T x_1} & y_2 y_2 \mathbf{x_2^T x_2} & \dots & y_2 y_p \mathbf{x_2^T x_p} \\ . & . & \dots & . \\ y_n y_1 \mathbf{x_p^T x_1} & y_n y_2 \mathbf{x_p^T x_2} & \dots & y_p y_p \mathbf{x_p^T x_p} \end{bmatrix} \lambda + (-\mathbf{1})^{\mathbf{T}} \lambda$$

Evaluation of the new sample

The discriminant function is given as

$$h(\mathbf{x}) = \sum_{\mathbf{i} \in \mathbf{SV}} \lambda_\mathbf{i} \mathbf{y_i} \mathbf{K}(\mathbf{x_i}, \mathbf{x}) + \mathbf{b}$$

Thus we take new sample **x** and compute the value of above formula. If it is positive, then it belongs to “+1 class” if it is negative then it belong to “-1 class”.

For computation we use all the support vectors (SV), i.e. the samples from training set that defines the margin.

Polynomial kernel

A polynomial mapping is a popular method for non-linear modelling. The features are transformed to Q-dimensional space and the dot product is computed. This can be compute directly via kernel:

$$K\left(\mathbf{x_i}, \mathbf{x_j}\right) = \left(1 + \mathbf{x_i^T x_j}\right)^Q$$

Note that the dot product is computed in original dimension.

The Q value can be 1, 2, … up to some finite number. This is set by user when specifying the kernel type.

(Gaussian) Radial Basis Function

We can also go to infinite dimensions… so we can compute the dot product for infinite dimension via kernel trick. How is it possible? Consider this Gaussian function:

$$K\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\frac{\left(\mathbf{x_i}-\mathbf{x_j}\right)^2}{2\sigma^2}}$$

Consider the simple 1D feature vector and ignore σ we can rewrite this function using Taylor series:

$$K\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\frac{\left(\mathbf{x_i}-\mathbf{x_j}\right)^2}{2\sigma^2}} = e^{-x_i^2}\, e^{-x_j^2} \sum_{k=0}^{\infty} \frac{2^k x_i^k x_j^k}{k!}$$

(Gaussian) Radial Basis Function

We can explicitly write few terms of the sum and see what happens:

$$K(\mathbf{x_i}, \mathbf{x_j}) = e^{-x_i^2} e^{-x_j^2} \sum_{k=0}^{\infty} \frac{2^k x_i^k x_j^k}{k!} =$$

$$e^{-x_i^2} e^{-x_j^2} \cdot \left(1 + 2x_i x_j + 2x_i^2 x_j^2 + \frac{8}{6} x_i^3 x_j^3 + \dots\right)$$

The $x_i x_j$, $x_i^2 x_j^2$, $x_i^3 x_j^3$ … represents the “coordinates” in the new high dimensional space.

Radial Basis Function (RBF)

Typically, the RBF in this form are used in SVM algorithms:

$$K\left(\mathbf{x_i}, \mathbf{x_j}\right) = e^{-\gamma\left(\mathbf{x_i} - \mathbf{x_j}\right)^2}$$

Parameter γ controls behavior of the separating hyperplane. If *γ* is small the decision plane will depend mainly on the points which are very close to this boundary and ignore the the points, which are far away. And *vice versa* - small γ will cause that points which are far away also influence the boundary.